# Phase -1.8 — Database Design & Entity Relationship Diagram (ERD)

## Objective

The API contract defines **how clients interact with the application**.

The database design defines **how the application stores and manages data**.

A well-designed database should:

- Accurately represent the business domain.
- Maintain data integrity.
- Prevent invalid data.
- Support efficient queries.
- Scale as the application grows.
- Minimize redundancy.
- Enforce business rules through constraints.

Before creating any Prisma models or writing SQL, we must first model the business domain.

---

# Database Design Principles

The database should follow these principles.

## Single Source of Truth

Every piece of business information should have exactly one authoritative location.

Example

- User information belongs in the `users` table.
- Event information belongs in the `events` table.

Avoid duplicating data across tables unless denormalization is intentional.

---

## Data Integrity

The database should prevent invalid data.

Examples include:

- A booking cannot exist without a user.
- A seat cannot exist without an event.
- A payment cannot exist without a booking.
- Duplicate emails should not be allowed.
- Invalid foreign keys should never exist.

Where possible, these rules should be enforced by database constraints rather than application logic alone.

---

## Normalization

The schema should initially target **Third Normal Form (3NF)**.

Benefits include:

- Reduced redundancy.
- Easier updates.
- Improved consistency.
- Better maintainability.

Performance optimizations through denormalization should only occur after measurement.

---

# Core Entities

The Version 1 system contains the following entities.

```
User

Role

RefreshToken

Event

Seat

Booking

Payment
```

Each entity maps directly to a business concept.

---

# Entity Overview

## User

Represents a registered customer or administrator.

### Responsibilities

- Authentication
- Authorization
- Booking ownership

### Relationships

- One User can have many Refresh Tokens.
- One User can create many Bookings.
- One User belongs to one Role.

---

## Role

Represents the permissions assigned to a user.

Version 1 roles:

- USER
- ADMIN

Keeping roles separate allows additional roles to be introduced without redesigning the schema.

---

## RefreshToken

Stores hashed refresh tokens.

Each token belongs to one user.

Revoked tokens remain stored for auditing and replay protection.

---

## Event

Represents an event available for booking.

Examples:

- Concert
- Movie
- Conference
- Sports Event

Each event owns its seating arrangement.

---

## Seat

Represents one physical seat belonging to one event.

Every seat has exactly one status.

Version 1 states:

```
AVAILABLE

HELD

BOOKED
```

---

## Booking

Represents a reservation made by a customer.

A booking links together:

- User
- Seat
- Payment

Booking lifecycle:

```
PENDING

CONFIRMED

FAILED

EXPIRED
```

```
CANCELLED
```

---

### Payment

Represents payment information for a booking.

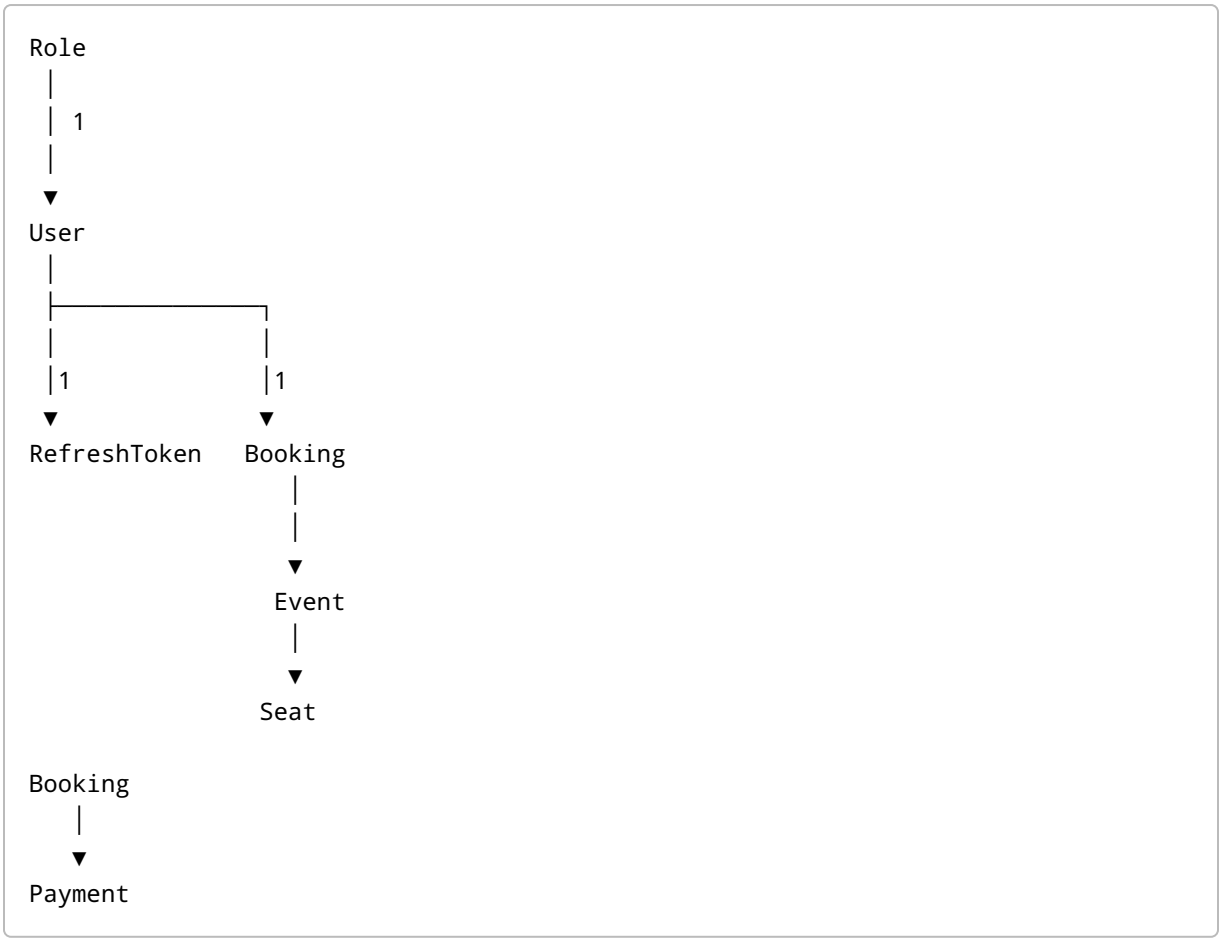
Version 1 uses a mock payment provider.

Future payment gateways should integrate without changing the core booking model.

---

# Entity Relationships

The following relationships exist.




---

# Cardinality

## Role → User

One Role

↓

Many Users

Relationship

```
Role 1 ──────────< User
```

---

## User → RefreshToken

One User

↓

Many Refresh Tokens

Relationship

```
User 1 ──────────< RefreshToken
```

Multiple active sessions should be supported.

---

## User → Booking

One User

↓

Many Bookings

Relationship

```
User 1 ──────< Booking
```

---

## Event → Seat

One Event

↓

Many Seats

Relationship

```
Event 1 ──────< Seat
```

---

## Seat → Booking

One Seat

↓

Many Bookings over time

Only one active booking may exist at any moment.

Historical bookings are preserved.

Relationship

```
Seat 1 ──────< Booking
```

---

## Booking → Payment

Version 1

One Booking

↓

One Payment

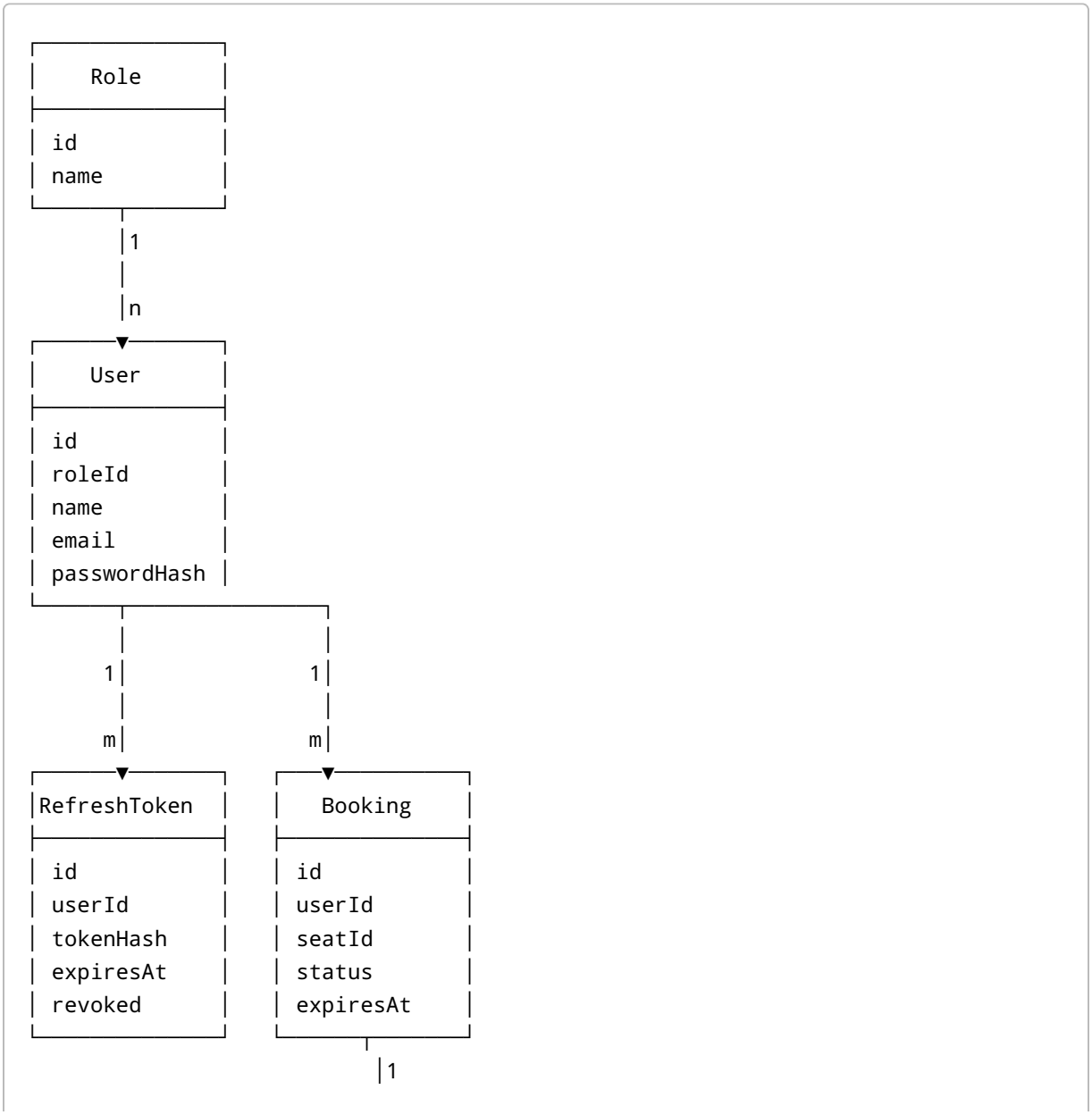
Relationship

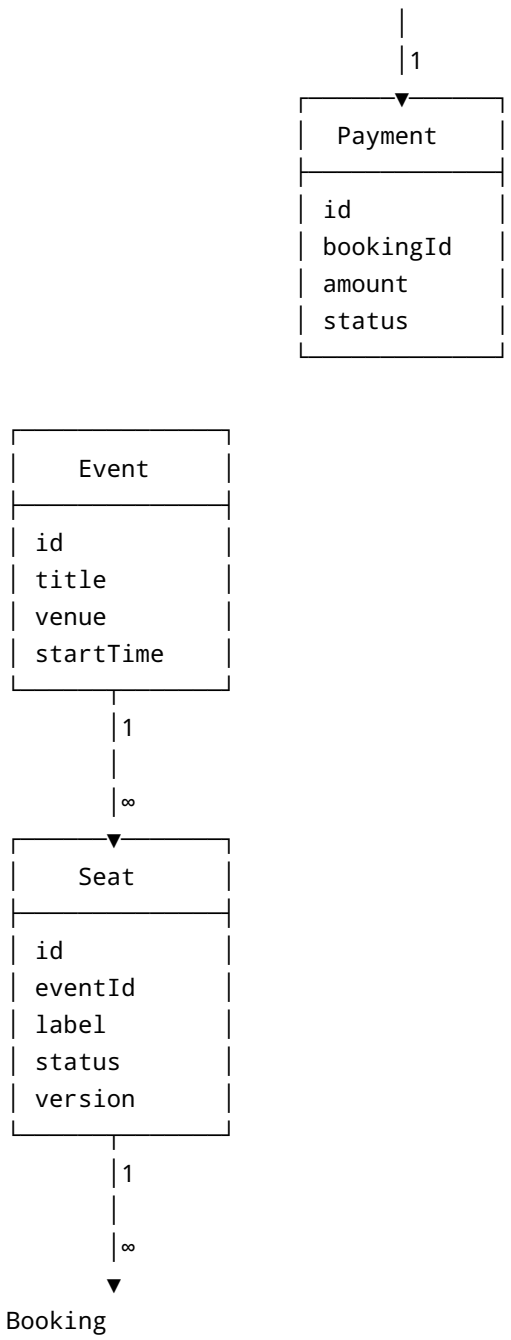
```
Booking 1 ──────────1 Payment
```

Future versions may support multiple payment attempts.

---

# Complete ER Diagram






---

# Primary Keys

Every table should use a surrogate primary key.

```
id UUID
```

Reasons:

- Globally unique.
- Difficult to guess.
- Suitable for distributed systems.
- Easy future migration to microservices.

---

# Foreign Keys

| Table | Foreign Key | References |
|---|---|---|
| User | roleId | Role.id |
| RefreshToken | userId | User.id |
| Seat | eventId | Event.id |
| Booking | userId | User.id |
| Booking | seatId | Seat.id |
| Payment | bookingId | Booking.id |

---

# Unique Constraints

The following fields must be unique.

| Entity | Field |
|---|---|
| User | email |
| Role | name |
| RefreshToken | tokenHash |

Future versions may add additional unique constraints.

---

# Indexing Strategy

Indexes should be added for frequently queried columns.

Initial indexes:

- User.email
- Booking.userId
- Booking.status
- Seat.eventId
- Seat.status
- Event.startTime
- RefreshToken.userId

Additional indexes should only be introduced after profiling query performance.

---

# Soft Deletes

Version 1 will use soft deletes for business entities.

Tables should contain:

```
deletedAt
```

instead of permanently deleting records.

Benefits:

- Audit history.
- Recovery of deleted records.
- Referential integrity.
- Safer production operations.

Reference data such as roles may continue using hard deletes where appropriate.

---

# Audit Fields

Every business entity should include common audit fields.

```
createdAt

updatedAt

deletedAt
```

Benefits:

- Easier debugging.
- Historical tracking.
- Compliance.
- Analytics.

---

# Future Database Extensions

The schema should support future additions without major redesign.

Examples include:

- Venue
- Organizer
- Ticket
- Coupon
- SeatCategory
- WaitingList
- PaymentAttempt
- Notification
- AuditLog

The current design intentionally leaves room for these future entities.

---

# Deliverables

At the completion of this phase, the application's data model is fully defined.

The team now understands:

- Every entity.
- Every relationship.
- Primary keys.
- Foreign keys.
- Cardinality.
- Constraints.
- Indexing strategy.
- Normalization approach.
- Audit strategy.
- Future extensibility.

The next phase, **Phase -1.9 — Sequence Diagrams & Business Workflows**, will define the runtime interaction between the client, application, database, cache, and background workers for each major use case.